# DBSCAN and Hierarchical Clustering

GENERAL ASSEMBLY

# Learning Objectives

By the end of this lesson, students should be able to:

- Describe the effect of **`epsilon`** and **`min_points`** on DBSCAN.
- Identify advantages and disadvantages of DBSCAN.
- Implement DBSCAN using **`sklearn`**.
- Describe two hierarchical clustering techniques.
- Implement agglomerative clustering using **`sklearn`**.

 © 2019 General Assembly

# Review



In your own words, what is **unsupervised learning** and why do we do it?

# k-Means Clustering

In unsupervised learning, one strategy is to **cluster** observations into **groups**.

Observations of the same group are more similar than observations in different groups.

We've learned one clustering algorithm so far: **k-means clustering**.

 © 2019 General Assembly

# k-Means Clustering



What are the pros and cons of k-means clustering?

# DBSCAN

There's another method of clustering that can sidestep some of the disadvantages of k-means: **DBSCAN**

DBSCAN stands for **D**ensity-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise

 © 2019 General Assembly

# DBSCAN

There's another method of clustering that can sidestep some of the disadvantages of k-means: **DBSCAN**

DBSCAN stands for **D**ensity-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise.

We detect areas of high density and low density:

- We cluster in areas of high density
- We avoid clustering in low-density areas

 © 2019 General Assembly

# Hyperparameters

DBSCAN requires you to specify two hyperparameters:

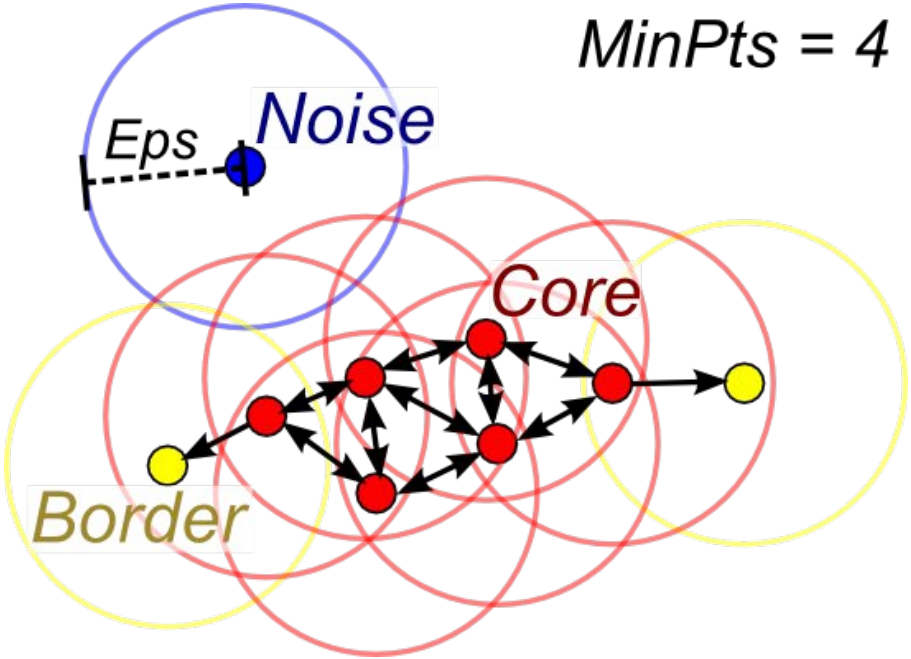
1. **`min_samples`**: the minimum number of points needed to form a cluster
2. **`epsilon`**: the “searching” distance when attempting to build a cluster

# Visualizing DBSCAN

A picture is worth a thousand algorithms:

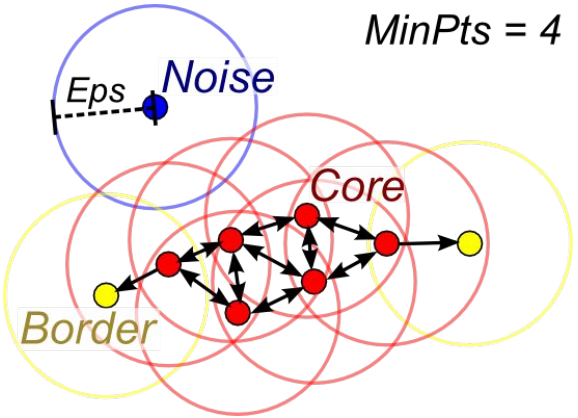
https://www.naftaliharris.com/blog/visualizing-dbscan-clustering/

# Visualizing DBSCAN



# Visualizing DBSCAN

- **Core points**: Points inside a cluster that have at least **`min_samples`** points within **`epsilon`**.
- **Border points**: Points inside a cluster that do not have at least **`min_samples`** points within **`epsilon`**
- **Noise**: Points that belong to no cluster




 © 2019 General Assembly

# Why DBSCAN?

DBSCAN allows us to detect some cluster patterns that k-means might not be able to detect. k-means wants sphere-ish blobs, where **DBSCAN wants densely connected blobs of any contiguous shape.**

We don't need to pre-specify the number of clusters; **DBSCAN will determine how many clusters are appropriate given fixed min_samples and epsilon values**. This is particularly valuable when we are clustering data in more than two or three dimensions.

Not every point is clustered! Good for identifying outliers.

# DBSCAN Pros and Cons

| Pros of DBSCAN | Cons of DBSCAN | k-means… |
|---|---|---|
| It handles blobs of *any* contiguous shape! | …but we need the data to have approximately uniform density throughout. | Can only handle spheroid blobs. |
| It tells us how many clusters there are! | …assuming we picked its ***two (2!)*** tuning parameters appropriately. | We have to select *k*. (But at least there's only one tuning parameter!) |
| It finds outliers for us! | …but if we picked our tuning parameters poorly, it may look like there are too many outliers. | Enforces everything to be in *some* cluster — even if it doesn't make sense! |

# Let's try it out!




 © 2019 General Assembly

# Hierarchical Clustering Techniques

# Hierarchical Clustering

Another class of clustering algorithms is called **hierarchical clustering**, and they can be divided into two subcategories:

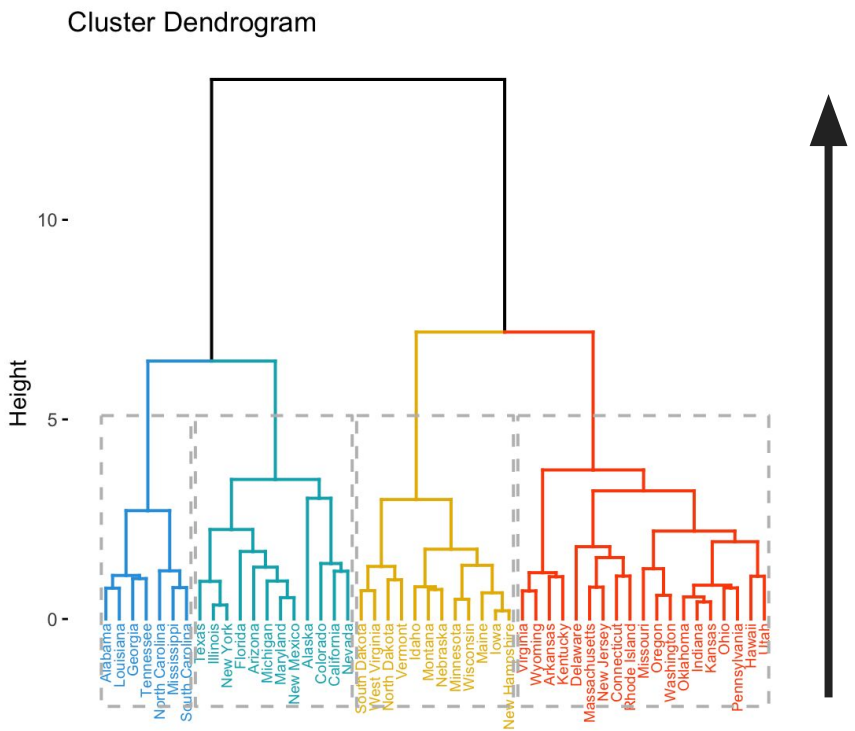
- **Agglomerative clustering**: Start with every observation in its own cluster, and merge them into bigger clusters one-by-one until some desired number of clusters is achieved.
- **Divisive clustering**: Start with every observation in one big cluster, and slowly split clusters apart until some desired number of clusters is achieved

# Hierarchical Clustering

Another class of clustering algorithms is called **hierarchical clustering**, and they can be divided into two subcategories:

- **Agglomerative clustering**: Start with every observation in its own cluster, and merge them into bigger clusters one-by-one until some desired number of clusters is achieved.
- ***Divisive clustering***: *Start with every observation in one big cluster, and slowly split clusters apart until some desired number of clusters is achieved*

# Agglomerative Clustering Dendrogram




Agglomerative clustering involves starting at the bottom and merging clusters until All is One.

# Agglomerative Clustering

In `sklearn`, you get to tune two relevant arguments:

- **`n_clusters`**: The desired number of clusters in your result
- **`linkage`**: The metric used to merge clusters
  - **`ward`** (default): Minimizes the variance of clusters being merged
  - **`single`** (popular): Uses the minimum distances between each point and each cluster. Using this option results in a special algorithm called **single-linkage clustering**.

 © 2019 General Assembly

# Let's try it out!

